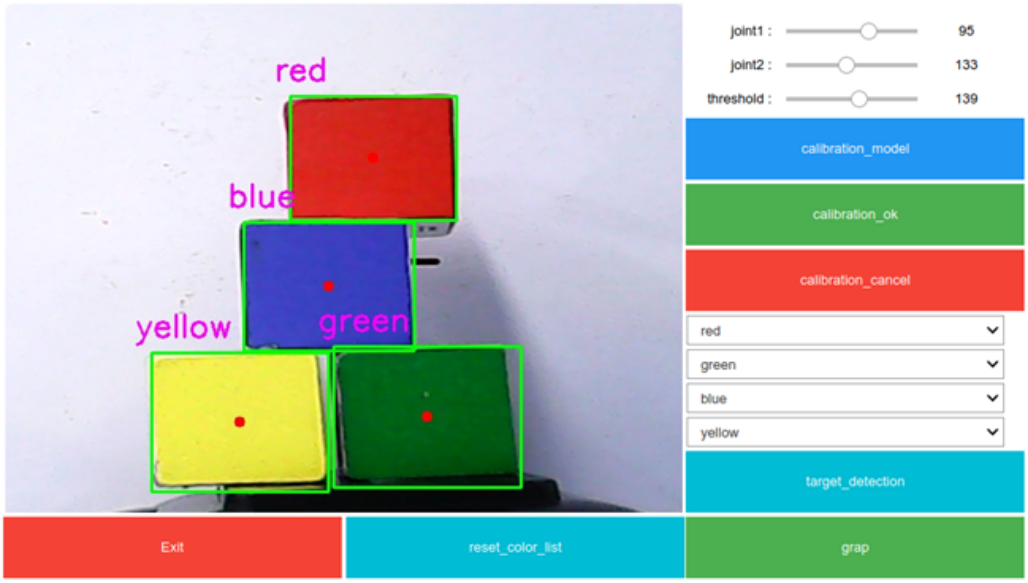
Color Sorting And Stacking

Gameplay Introduction

```
ros2 run dofbot_info dofbot_server # Start server node
Color sorting gameplay path: dofbot_ws/src/dofbot_color_identify/scripts/颜色分拣
color_sort.ipynb
Color stacking gameplay path: dofbot_ws/src/dofbot_color_stacking/scripts/颜色堆叠
color stacking play.ipynb
```



(1)HSV Calibration: Since different environments produce different color recognition results, it is best to perform HSV calibration before playing color sorting. For specific methods, refer to [Color Calibration].

(2)Frame Calibration: After color calibration is complete, frame calibration needs to be performed to obtain the position information of blocks within the frame. For specific calibration methods, refer to [Visual Positioning].

(3)Select Color: Players can freely choose the color to grab according to their needs. The robotic arm will sort according to the color selected by the player.

(4)Grab Sorting: After the camera recognizes the block, click the grab button to sort it.

(5)Clear Color: If the player wants to change the selected color, simply clear the selected color and reselect for recognition.

(6)Exit Program: To close this gameplay, click the exit button to release resources.

Color Recognition

Note: Effective only after frame calibration is complete.

- Operation process

(1) After calibration is complete, select a color in the color option box on the right side (no color selected by default), as shown in the figure above.

(2) After color selection is complete, click the [target_detection] button to call the color recognition function to mark the detected blocks.

(3) Click the [grap] button to sort out the recognized blocks.

(4) Click the [reset_color_list] button to clear the selected color sequence. The color selection bar still shows the previous selection order, just replace it. Repeat steps (2), (3).

- Code design

Color selection sequence, generate color sequence after selection is complete

```
def color_list_one_Callback(value):
    pass

def color_list_two_Callback(value):
    pass

def color_list_three_Callback(value):
    pass

def color_list_four_Callback(value):
    pass

color_list_one.observe(color_list_one_Callback)
color_list_two.observe(color_list_two_Callback)
color_list_three.observe(color_list_three_Callback)
color_list_four.observe(color_list_four_Callback)
```

Click the [target_detection] button to call the color recognition function select_color()

```
def select_color(self, image, color_hsv, color_list):
    '''
    Select recognition color
    :param image: Input image
    :param color_hsv: Example {color_name:((lowerb),(upperb))}
    :param color_list: Color sequence:['0': none   '1': red '2': green '3': blue
'4': yellow]
    :return: Output processed image, (color, position)
    '''
    pass
```

Parse the passed color sequence and color information. Detect blocks in the color sequence respectively. Return block coordinates

```
def get_Sqaure(self,color_hsv):
'''
Color recognition, get block coordinates
'''
pass
```

For detailed code introduction, see dofbot_ws/src/dofbot_color_identify/scripts/颜色分拣color_sort.ipynb

Get Inverse Solution Results

- Operation process

After detecting the block, click the [grap] button to grab

- Code design

Send the obtained block coordinate information through ROS server communication. Solve the angle each joint should rotate through inverse kinematics. Obtain joint angles to drive the robotic arm to grab.

```
def server_joint(self,posxy):
    '''
    Publish position request, get joint rotation angle
    :param posxy: Position point x, y coordinates
    :return: Rotation angle of each joint
    '''
    # Wait for server to start
    self.client.wait_for_service()
    # Create message packet
    request = kinemaricsRequest()
    request.tar_x = posxy[0]
    request.tar_y = posxy[1]
    request.kin_name = "ik"
    try:
       # Get inverse solution response result
       response =   self.client.call(request)
        pass
  except Exception:  rospy.loginfo("arg error")
```

For detailed code, see the identify_target.py and identify_grap.py under dofbot_ws/src/dofbot_color_identify/scripts/

- Grab design process

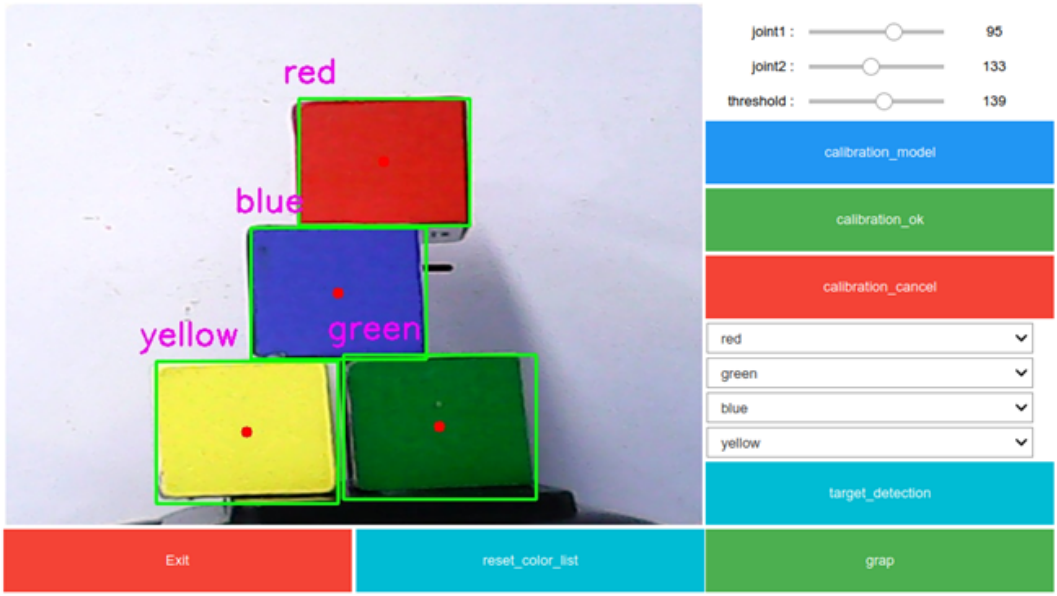
Lift robotic arm -> Release gripper -> Move to block position -> Grip -> Lift -> Move to target position -> Release gripper -> Reset robotic arm.

Color Stacking

Color stacking gameplay is similar to color sorting, except that the robotic arm stacks the blocks vertically according to the selected color at the end.

Note: Effective only after frame calibration is complete. Color stacking not only requires the ability to grip the blocks, but the gripping position should be as consistent as possible. The positioning requirement is very high. Users can adjust the gripping position and pose according to actual situations. For specific adjustment methods, please refer to the [Visual Positioning] section.

- Example image



- Operation process

(1) After calibration is complete, select a color in the color option box on the right side (no color selected by default), as shown in the figure above.

(2) After color selection is complete, click the [target_detection] button to call the color recognition function to mark the detected blocks.

(3) Click the [grap] button to sort the recognized blocks, then stack them into the gray frame. The stacking order is the same as the color selection order in the figure above.

(4) Click the [reset_color_list] button to clear the selected color sequence. The color selection bar still shows the previous selection order, just replace it. Repeat steps (2), (3). For detailed code introduction, see dofbot_ws/src/dofbot_color_stacking/scripts/颜色堆叠color stacking play.ipynb